Modelo de Entidad-Relación

Introducción

En este documento discutiremos una muy buena estrategia para el diseño de bases de datos (correspondiente al modelo de datos conceptual): el Modelo de Entidad Relación.

Existen varios diagramas para implementar gráficamente este modelo. El primero fue el propuesto por Peter Chen, "*The Entity–Relationship Model: Toward a Unified View of Data*" publicado en ACM Transaction on Database Systems, en 1976.

Luego, surgieron otros:

- La extensión con modelado para objetos introducida por Toby Teorey, Dongqing Yang y James Fry, *"A Logical Design Methodology for Relational Databases Using the Extended Entity-Relationship Model",* publicada en Computing Surveys, en 1986 (ver lecturas en IOL).
- Muchas otras.

En este apunte desarrollaremos la notación basada en Chen y su extensión para objetos.

1. Diseño de una Base de Datos

Habiendo resaltado que las **Bases de Datos son parte de los Sistemas de Información** que maneja una organización, vamos a centrarnos en ellas para sacarles el máximo provecho.

Como citamos previamente, es muy importante entender a los datos por medio de un **modelo** que permita generar abstracciones. Hemos categorizado a los **modelos de datos** (sección 4 del apunte anterior) según nivel de abstracción en: **Conceptual** (muy alto nivel), de **Implementación** (nivel intermedio) y **Físico** (bajo nivel).

Los datos vistos en el **nivel de abstracción más alto** sirven para **realizar diseños de bases de datos**. Nosotros en particular analizaremos el **modelo conceptual más usado**: el **Modelo de Entidad/Relación (Entity-Relationship Model)** y su extensión.

Un buen diseñador comienza siguiendo los siguientes pasos:

- **Reúne los requerimientos del sistema**. Primero se debe conversar con los usuarios finales del sistema con el fin de conocer sus expectativas respecto del mismo. Hay que averiguar las otras partes del sistema de información que interactuarán con el sistema de base de datos. En esta etapa hay que recolectar: información de los usuarios finales del sistema que nos contarán los datos que manipulan, nos mostrarán formularios existentes, manuales de procedimientos en uso o a aplicar, etc. La salida de esta etapa es un documento escrito con los requerimientos surgidos: información a manipular y requerimiento funcional de la aplicación (qué esperan que haga el sistema).

Silvia Alicia Gómez & Leticia Irene Gómez

- A partir de los requerimientos **crea un esquema conceptual** para la base de datos por medio de un modelo de datos conceptual. Este esquema incluye descripción de la estructura de los datos, relaciones y restricciones, y por **no poseer detalles de implementación** puede ser compartido con los futuros usuarios del sistema (no necesariamente técnicos). A partir de este esquema las operaciones **básicas** de modelo de datos pueden utilizarse para especificar transacciones de alto nivel correspondiente a los requerimientos funcionales. Nosotros utilizaremos en esta etapa de diseño conceptual el **modelo de entidad-relación (extendido)**. Esta etapa es temprana y todavía no se conoce ni se puede referenciar a un DBMS en particular. La idea es precisamente no influenciarse en el diseño por ninguna característica de un DBMS en particular.

- **Realiza la implementación** de la base de datos con algún DBMS comercial/open source. La elección del DBMS específico responde a distintas motivaciones: técnicas, corporativas, económicas, etc. Según cuál sea el DBMS elegido se utilizará el modelo de implementación correspondiente (relacional, jerárquico, red u objeto). Existen procedimientos para realizar, en forma genérica, el mapeo del *esquema conceptual* (obtenido en la etapa anterior) al *esquema de implementación* elegido, pudiendo luego adaptar lo obtenido al DBMS concreto elegido con la intención de aprovechar al máximo sus características. Esta etapa se denomina **diseño lógico de la base de datos** y su salida corresponde a una especificación DDL para el esquema conceptual de la base de datos. Ejemplo: si en esta etapa se decidió elegir Informix Dynamic Server 7.3 que es relacional, entonces se toma el diagrama de entidad/relación y genéricamente se lo pasa al esquema del modelo relacional. Si se desea, se le pueden hacer cambios para aprovechar las ventajas específicas que ofrece Informix Dynamic Server 7.3 (o no, si se piensa migrar más tarde a otro DBMS).

- Luego, teniendo en cuenta la performance buscada, procede al **diseño físico de la base de datos**, o sea se incorporan índices que agilicen la búsqueda de la información, junto con el desarrollo de las aplicaciones que la manejen.

- Finalmente realiza o supervisa la **carga de los datos** necesarios y **examina el funcionamiento** del sistema, pudiendo realizar correcciones si fuera necesario.

1.1 Modelo de Entidad/Relación (Entity/Relationship Model)

Como citamos previamente es el que elegimos para modelar el mini-universo a representar. Percibe al mundo como un conjunto de objetos básicos llamados *entidades* (*entities)*) y un conjunto de *relaciones* (*relationships*) entre los mismos. Como esta etapa procede tempranamente, esta herramienta nos permitirá representar entidades, relaciones y atributos involucrados **sin referenciar ninguna característica específica de DBMS alguno**.

La primera propuesta fue realizada por P. Chen en 1976, pero luego fue modificado por otros autores. Maneja tres nociones básicas: conjunto de **entidades**, conjunto de **relaciones** y **atributos**.

1.1.1 Conjunto de Entidades (Entity Set o Entity Type)

Una **Entidad** es cualquier "cosa" del mundo real (concreta o abstracta) que puede verse como los objetos básicos de un dominio de interés. Ejemplo: un empleado en particular, la materia "Base de Datos" en el ITBA. Cada entidad tiene un conjunto de propiedades que la describen y se llaman **atributos**, por ejemplo, el sueldo del empleado, el nombre de la materia. Por otra parte,

Silvia Alicia Gómez & Leticia Irene Gómez

cada entidad posee un valor para cada uno de sus atributos. Ejemplo: esta materia posee para el atributo *nombre* al valor 'Base de Datos', en cambio, otra materia posee para el mismo atributo el 'Estructura de Datos y Algoritmos'.

Un **Conjunto de Entidades** es la colección de todas las entidades que poseen los mismos atributos (no hablamos de sus valores). Por ejemplo, el conjunto de entidades *materia* está definido por todas las materias particulares. Suelen denominarse con nombres en singular, ya que representan un conjunto.

Un conjunto de entidades describe el **esquema** o **intensión** para ese conjunto de entidades que comparten la estructura. Cada una de las entidades individuales de un cierto conjunto de entidades agrupadas en colección forma la **extensión**. Es análogo a cuando se define un conjunto por definición (enumerando sus propiedades) o por extensión (enumerando sus componentes).

Representar en el modelo *los conjuntos de entidades* en vez de *las entidades* simplifica notablemente el análisis, ya que el número es mucho menor y resulta más sencillo focalizarse en sus características.

Las entidades **se diferencian entre sí** dentro del conjunto. Los atributos pueden ser de dos tipos: identificatorios o descriptivos. Los primeros son las que aseguran las diferencias entre entidades o sea individualizan a una entidad (ver *atributo* en la sección 2.1.3). Los otros sólo ofrecen características.

Los tipos de entidades pueden ser **fuertes** (*strong entity type*) o **débiles** (*weak entity type*), según discutiremos más adelante.

1.1.2 Conjunto de Relaciones (Relationship Set o Relationship Type)

Una **Relación** es una asociación entre entidades particulares. Un **Conjunto de Relaciones** es la colección de todas las relaciones del mismo tipo. Formalmente sean E_1, E_2,...E_n conjuntos de entidades con $n \geq 2$, un conjunto de relaciones R es un **subconjunto del producto cartesiano** $E_1 \times E_2 \times ... \times E_n$.

Por ejemplo, se tiene el conjunto de entidades *alumno* y el conjunto de entidades *materia*. El conjunto de relaciones *esta_inscripto* entre ambos asocia la inscripción de alumnos en las materias. Una relación sería una asociación entre un alumno en particular y una materia específica.

Se denomina **grado o aridad** a la cantidad de conjuntos de entidades participantes. Cuando se asocian dos conjuntos de entidades se dice que es binaria, si se asocian tres se denomina ternaria. Por ejemplo, el conjunto de relaciones *está_inscripto* es binario porque asocia a dos conjuntos de entidades participantes: *materia y alumno*.

Aclaración

La **función que desempeña** una entidad en una relación recibe el nombre de **rol**. En general los conjuntos de entidades participantes en un conjunto de relaciones son diferentes y por lo tanto sus roles son implícitos (no hace falta indicarlos explícitamente). Cuando no son distintas es preciso indicar los papeles explícitamente para resaltar cómo participa determinada entidad en determinada relación.

Silvia Alicia Gómez & Leticia Irene Gómez

Conjunto de Relaciones Recursivas

Un conjunto de relaciones se dice que es *recursiva* cuando el **mismo** conjunto de entidades **participa más de una vez** en el conjunto de relaciones **con distintos roles**. Por convención a las relaciones recursivas se las considera ***unarias***.

Ejemplo:

El conjunto de relaciones *es_jefe* es recursiva e involucra dos veces al conjunto de entidades *empleado.* La primera entidad que aparezca en una relación es el jefe y la segunda es el subordinado. Esto debe señalarse porque un empleado en particular puede ser jefe de alguien y subordinado de otro.

El modelo de Entidad-Relación es un lenguaje gráfico que permite representar con ciertas restricciones la semántica de la aplicación. Es muy sencillo y permite capturar una primera aproximación para describir ciertas características de una aplicación. Cuanto más se pueda decir en esta etapa (cuanto más se capte en ese modelado) mejor, ya que permite introducir en forma temprana restricciones semánticas. Si embargo, no todas las restricciones pueden expresarse con este modelo. Hay cosas que pueden especificarse y otras que no (debiéndose posponer para el modelado lógico). Las **restricciones que pueden imponerse a los conjuntos de relaciones** (surgen del mini_universo) son:

- **Cardinalidad**

Indica el número de relaciones en las que una entidad puede participar (con cierto rol). Dicho de otra forma indica el número de veces en que puede aparecer una entidad con otras en el conjunto de relaciones. En forma general, podemos decir que en una relación n-aria la cardinalidad limita el número de ocurrencias que las entidades pueden ocurrir en cierta relación con respecto a las restantes n-1 componentes de la relación.

Ejemplo:

Si tenemos un conjunto de relaciones R entre dos conjuntos de entidades A y B, o sea en una **relación binaria la cardinalidad** puede ser:

- **1:1 (uno a uno)**: una entidad en A puede asociarse a lo sumo con una entidad en B, y viceversa.
- **1**:**N (uno a varios)**: una entidad en A puede asociarse con muchas entidades en B, pero una entidad en B puede asociarse con a lo sumo una entidad en A.
- **N**:**1 (varios a uno)**: una entidad en A puede asociarse con a lo sumo una entidad en B, pero una entidad en B puede asociarse con muchas entidades en A.
- **M**:**N (varios a varios)**: una entidad en A puede asociarse con muchas entidades en B, y una entidad en B puede asociarse con muchas entidades en A.

Podemos imaginar ejemplos concretos, como ser:

a) Si un *personal trainer* sólo puede tener un alumno a su cargo pero no hay restricciones sobre la cantidad de *personal trainers* que puede tener el alumno, entonces la cardinalidad de ese conjunto de relaciones es N: 1

Silvia Alicia Gómez & Leticia Irene Gómez

b) Si un *personal trainer* puede tener un grupo de alumnos a su cargo, pero ninguno de ellos puede responder a más de un *personal trainer*, entonces la cardinalidad de ese conjunto de relaciones es 1:N

c) Si no hay restricciones respecto a las cantidades de alumnos que pueden tener los *personal trainers* y viceversa, entonces la cardinalidad de ese conjunto de relaciones es M:N

Ejemplo:

Si la **relación fuera ternaria** (entre 3 entidades participantes) la cardinalidad puede ser: 1:1:1, 1:1:N, 1:M:N, P:M:N (y permutaciones).

➢ **Participación o Existencia**

Especifica si la existencia de una entidad depende o no de su participación con otra entidad en una relación. Si la exigencia de existencia se cumple, entonces el conjunto de entidades que declara que existen sus entidades en cuanto participan de relaciones se dice que es *subordinada*, y el otro conjunto de entidades participante del conjunto de relaciones se llama *dominante (owner o dueño)*. La participación de conjunto de entidades en un conjunto de relaciones puede ser entonces *parcial* o *total* según su existencia dependa o no de la participación mencionada, respectivamente. Más formalmente, la participación del conjunto de entidades A es **total (full participation o mandatory participation)** en el conjunto de relaciones R si todas sus entidades tienen que **participar por lo menos de una relación**. La participación se dice **parcial** en caso contrario.

Ejemplo:

Si el ITBA exigiera que todo alumno esté por lo menos inscripto en una materia para seguir siendo alumno, entonces el conjunto de entidades *alumno* tendría una participación total en el conjunto de relaciones *esta_inscripto*.

1.1.3 Atributo (Attribute)

Cada propiedad que describe una entidad/relación se denomina **atributo**. Los atributos que modelan un conjunto de entidades/relaciones describen tempranamente el modelo (intensión). Cada atributo posee un conjunto de valores permitidos y se llama **Dominio**. Por ejemplo, el dominio del atributo sueldo del conjunto de entidades *empleado* son los reales mayores que cero, el dominio del atributo nombre del conjunto de entidades *materia* son caracteres alfanuméricos de cierta longitud. Una entidad/relación en particular toma valores para cada uno de sus atributos (extensión).

Formalmente un atributo de un conjunto de entidades/relaciones es una función que asigna al conjunto de entidades/relaciones en el conjunto de partes del dominio. A su vez como un conjunto de entidades/relaciones puede estar caracterizado por varios atributos, cada entidad/relación en particular puede describirse por un conjunto de pares (*atributo*, valor). Por ejemplo, el conjunto de entidades *materia* está caracterizado por los atributos *nombre y código* entonces esta materia en particular puede ser descripta por el conjunto { (*nombre*, 'base de datos´), (*código*, '72.37') }.

Cabe señalar que los conjuntos de relaciones también pueden poseer atributos. Por ejemplo, el conjunto de relaciones *aprobado* que relaciona el conjunto de entidades *alumno* con el conjunto de entidades *materia* y representa la nota del final que sacó el alumno cuando aprobó la materia está caracterizado por el atributo *nota* donde para un alumno y materia en particular tendrá un valor

Silvia Alicia Gómez & Leticia Irene Gómez

mayor o igual que 4 (si es que aprobó). Notar que el atributo *nota* no es de la entidad alumno (un alumno tiene diferentes notas, dependiendo de la materia en cuestión) y tampoco de la materia (una materia es aprobada con distintas notas, según el alumno que sea), por lo tanto dicho atributo caracteriza la relación *aprobado*, no las entidades intervinientes. En ese ejemplo, el conjunto de relaciones es M:N y los atributos deben ser especificados como parte del conjunto de relaciones (y no de los conjuntos de entidades participantes). Sin embargo, en un conjunto de relaciones con cardinalidad 1:N, N:1 ó 1:1, los atributos pueden migrarse hacia alguno de los conjuntos de entidades participantes. En el caso 1:N debe ir hacia el conjunto de entidades del lado N de la relación, y en el caso 1:1 es indistinto. La decisión de dejar el atributo de un conjunto de relaciones en ella o migrarla (si se puede) hacia el conjunto de entidades lícito es totalmente una decisión que queda a cargo del diseñador del sistema de base de datos.

Un atributo en el modelo de entidad/relación puede clasificarse en:

- **Simple** vs **Compuesto (Simple vs Composite):** Los **simples o atómicos** son los que no precisan ser referenciados por partes. Ejemplo: el *sueldo* del conjunto de entidades *empleado,* el *nombre* del conjunto de entidades *materia* son atributos simples. Los **compuestos** son aquellos que forman una jerarquía y se usan cuando es preciso tanto referenciar al atributo como unidad como hacer referencia a cada una de sus partes por separado. Ejemplo: el atributo *dirección* del conjunto de entidades *empleado* puede estar compuesto por los atributos *calle, altura y departamento*. El valor de un atributo compuesto es la concatenación de los valores de las respectivas partes que lo forman.

- **Univaluado** vs **Multivaluado (Single-valued vs Multivalue):** Los **univaluados** son los que pueden tomar un único valor para determinada entidad. Ejemplo: el atributo *sueldo* de un empleado. Los **multivaluados** son los que pueden tomar más de un valor simultáneamente. Ejemplo: el atributo *color* es multivaluado para un auto ya que la entidad taxi tiene el valor negro y amarillo.

- **Almacenado y Derivado (Stored and Derived):** Los atributos que pueden calcularse en función de otros atributos y no precisan almacenarse se denominan **derivados**. Ejemplo: el atributo *antigüedad* para el conjunto de entidades *empleado* puede derivarse del atributo *fecha_de_ingreso.* Ejemplo: el atributo *cantidad_de_materias* puede deducirse contando la cantidad de entidades *materia*.

1.1.3.1. Atributo Clave

Existe una restricción muy importante que puede presentar un atributo: la de ser **clave** (mencionado previamente como identificatorio):

Atributo Clave (Key Attribute)

Una clave es un conjunto de uno o más atributos que tomados colectivamente permiten identificar en forma unívoca a una **entidad** en un conjunto de entidades. Un atributo clave NO puede usarse para identificar un conjunto de relaciones. Sólo sirve para entidades.

Silvia Alicia Gómez & Leticia Irene Gómez

Por ejemplo, el atributo *legajo* permite identificar unívocamente a cada uno de los alumnos en el conjunto de entidades *alumno*. El atributo *legajo* + *nombre* también es atributo clave (hay información de más).

Importante

Si se especifica en el diagrama de entidad/relación que un/os atributo/s es/son clave/s para un tipo de entidad se está asegurando que toda extensión de dicho tipo de entidad no poseerá valor repetido para dicho/s atributo/s.

Se está garantizando el cumplimiento de la restricción de la clave (key constraint) por adelantado. Todo conjunto de entidades debe tener su atributo clave.

Importante

En el modelo de entidad-relación un tipo de entidad puede poseer más de una clave. Con esto no estamos diciendo que una clave puede estar formada por varios atributos (cosa que también es lícita), sino que un tipo de entidad puede tener varias claves, cada una de ellas simple o compuesta, que identifique unívocamente a cada extensión.

Cabe señalar que si un conjunto de entidades arma su clave a partir de sus atributos se denomina **fuerte** (**strong entity type**). En contraposición encontramos conjuntos de entidades que tiene su clave formada por atributos propios más atributos de otros conjuntos de entidades. A este tipo se las denomina **débil** (**weak entity type**).

Un conjunto de entidades **débiles** seguro que tiene una **restricción de participación total** y posee una clave parcial (no posee atributo clave propio, pero combinado con otros atributos del conjunto de entidades con el que participa de cierta relación, sí forma clave). Sin embargo, no es cierto que todo conjunto de entidades que posea participación total sea débil.

Por ejemplo, el conjunto de entidades *licencia_para_conducir* posee participación total porque no existe entidad si no hay alguna entidad del conjunto de entidades *persona* asociada, pero no es débil (el atributo clave puede ser *código*)

Aclaración

Un conjunto de entidades débiles está indicando que son parte del conjunto de entidades fuertes con las que está asociada en la relación. O sea, permite representar el concepto "está formado por" o "agregación".

En general las entidades débiles pueden reemplazarse colocando directamente sus atributos en el otro conjunto de entidades que participa con él en cierto conjunto de relaciones. Si fuera 1:N serían atributos compuestos multivaluados. Representar un conjunto de entidades débiles o colocar sus atributos en el otro conjunto de entidades es más que nada una decisión que queda a cargo del diseñador del sistema de base de datos.

Silvia Alicia Gómez & Leticia Irene Gómez

1.2 Diagrama del Modelo de Entidad/Relación (Entity/Relationship Diagram)

Siempre las representaciones gráficas son más fáciles de entender, y sobre todo, pueden discutirse con los futuros usuarios del sistema de base de datos, que no necesariamente resultan ser técnicos del área.

A continuación desarrollaremos la representación de los conceptos vistos anteriormente, a través de las convenciones gráficas del diagrama que responde a la notación de Chen.

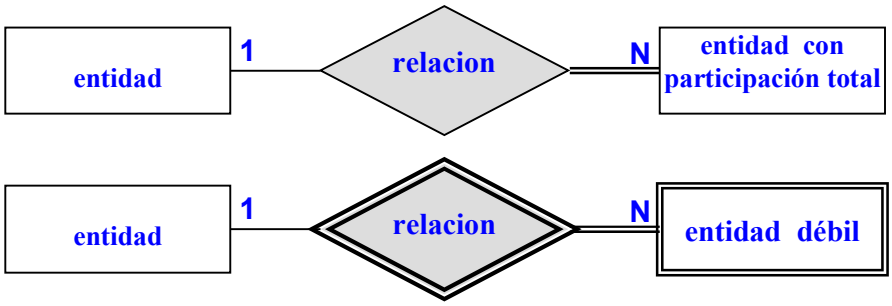
➢ *Conjunto de Entidades*

Un conjunto de **entidades** se representa por un **rectángulo** que posee dentro el nombre de la misma. Si fuera débil, el rectángulo debe ser hecho con doble línea.

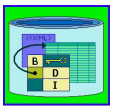
➢ *Conjunto de Relaciones*

Un conjunto de **relaciones** se representa por un **rombo** con su nombre dentro y se lo une por medio de líneas a los conjuntos de entidades participantes. La cardinalidad de un conjunto de relaciones se indica escribiendo 1 o N (o M o P) según corresponda a los lados del rombo.

Si la participación de un conjunto de entidades en un conjunto de relaciones fuera total se indica uniendo a la misma con la relación por medio de una línea doble. Si une alguna entidad débil y justo es debido a dicha relación que la misma es débil, se lo dibuja también con línea doble.

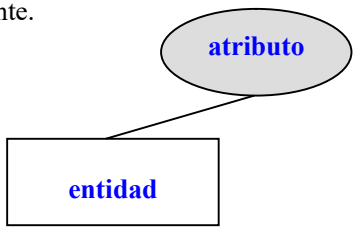


Silvia Alicia Gómez & Leticia Irene Gómez

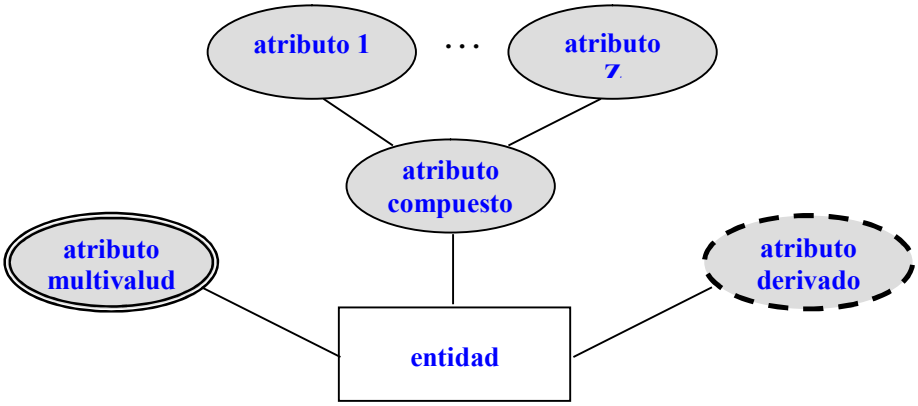


➢ *Atributos*

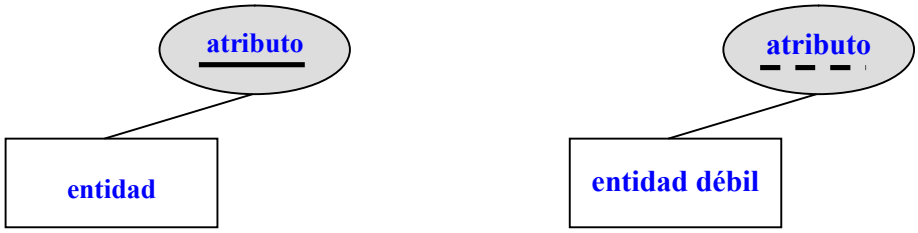
Los **atributos** se representan por medio de un **óvalo** con su nombre dentro, unido al conjunto de entidades o relaciones correspondiente.



Si fuera multivaluado se lo encierra en un óvalo doble. Si fuera compuesto se lo indica por medio de una estructura arborescente. Si el atributo fuera derivado (se puede calcular en base a otros datos) el óvalo se dibuja en línea punteada.



Si contiene un atributo clave el nombre del mismo se subraya. Si la clave fuera parcial (corresponde a una entidad débil) se la subraya con línea punteada.



Importante

Cada conjunto de entidades debe tener su clave. Si es una entidad débil su clave se arma con su clave parcial más los atributos claves del otro conjunto de entidades que participan del conjunto de relaciones.

Silvia Alicia Gómez & Leticia Irene Gómez

1.3 Modelo de Entidad/Relación Extendido: Representación de Jerarquías

Como se observó previamente, el modelo presenta la posibilidad de representar complejos sistemas. Sin embargo el modelo básico puede extenderse permitiendo representar mejor el mini-universo en cuestión. Muchos autores han presentado variantes al modelo original (no hay consenso al respecto): Navathe, Date, Elmasri, Teorey, etc. Nosotros tomaremos las propuestas por Toby J. Teorey, Dongqing Yang y James P. Fry.

En este modelo de entidad/relación extendido se permite expresar que un conjunto de entidades es subclase de otro conjunto de entidades debido a que posee mayor especialización. También podemos pensarlo inversamente y decir que un conjunto de entidades es superclase de otro conjunto de entidades porque posee mayor generalidad.

Por ejemplo, el conjunto de entidades *empleado* es superclase de los conjuntos de entidades *efectivo, contratado, free-lance.*

Obviamente una entidad que sea del conjunto de las entidades *efectivo* **es_una** entidad del conjunto de entidades *empleado*.

Muy Importante

Cuando se verifica especialización encontramos que el conjunto de entidades subclase hereda todos los atributos y participa de todas las relaciones de la superclase (ya que es un caso especial)

Existen dos tipos de jerarquías:

- **Generalización (Generalization):** Si se tiene un conjunto de entidades y se lo divide en subclases disjuntas.

  El diagrama que representa esto se lo hace representando a cada una de las subclases y uniéndolas a la superclase por una doble flecha, vía un hexágono que posea el atributo que produjo la separación.

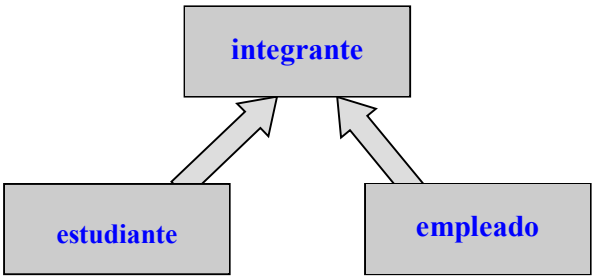
- **Es-un**: si sólo se especifica especialización, pero falla que al unir todas las subclases se obtenga la superclase, o se tiene que las subclases no son necesariamente disjuntas.

  El diagrama que representa esto es similar al anterior pero sin el hexágono del medio.

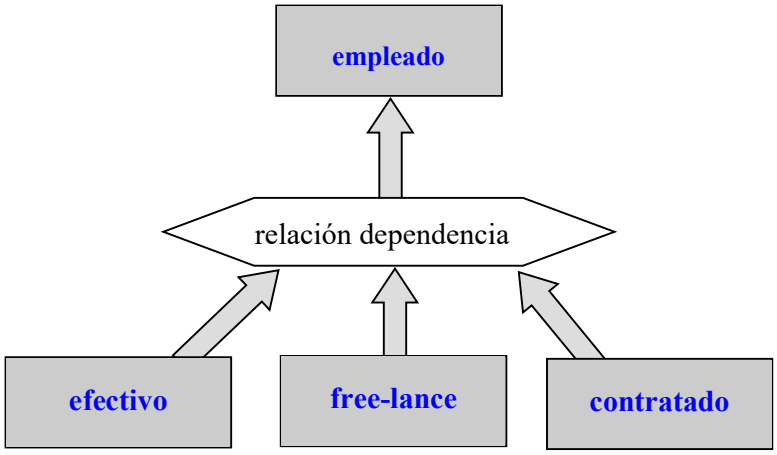
Ejemplos:

1) El conjunto de entidades *integrante* para esta universidad tiene por sublcases *estudiante* y *empleado*. Pero si un estudiante puede también ser empleado como es el caso de los operadores, entonces se trata de una jerarquía **ES-UN**:

Silvia Alicia Gómez & Leticia Irene Gómez



2) Si se considera que todos los empleados de una empresa pueden ser o bien efectivos, o bien contratados o bien free-lance pero no existe empleado que pueda pertenecer a más de una categoría, entonces se trata de una GENERALIZACIÓN y se representa así:



2. *Semántica de las Relaciones*

2.1 *Relaciones redundantes o derivadas*

Se dice que una relación es redundante o derivada si al eliminarse no se pierde información. La información puede seguir obteniéndose.

A simple vista, podría pensarse que si un EER que se representa por medio de un grafo posee un ciclo hay relaciones redundantes. Sin embargo, que exista un grafo con ciclo no significa que haya necesariamente relaciones redundantes.

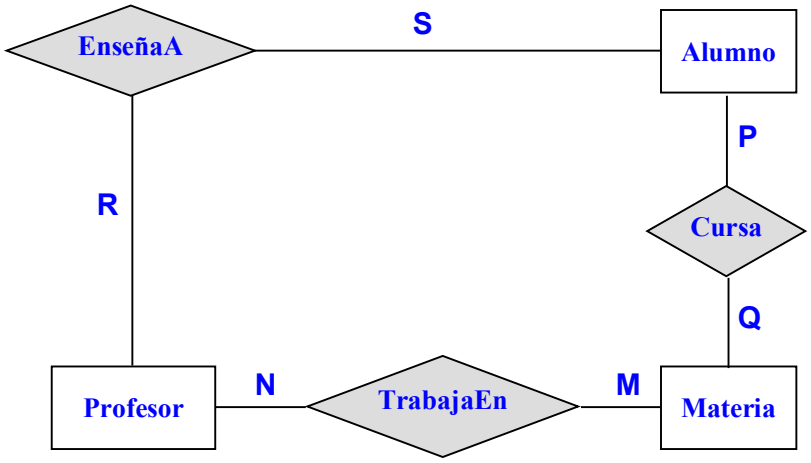
Importante

Las relaciones redundantes DEBEN eliminarse

Silvia Alicia Gómez & Leticia Irene Gómez

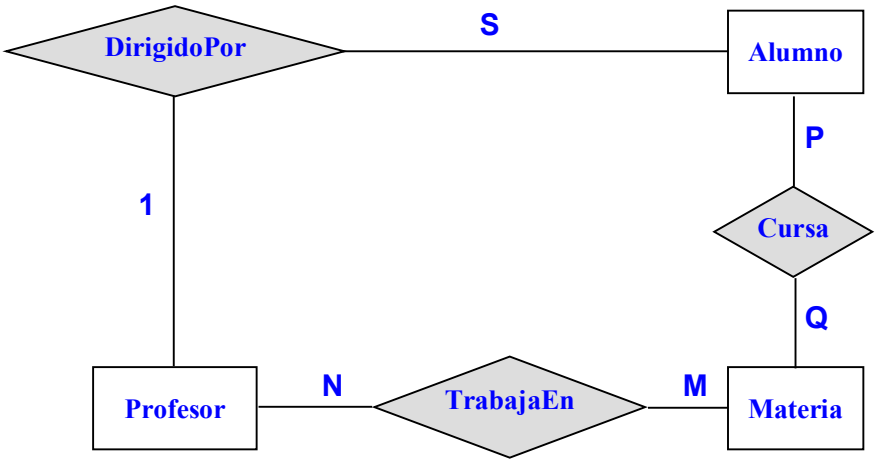
Ejemplo:

Entre el conjunto de entidades Alumno y Profesor, la relación ***EnseñaA*** es redundante. Debe eliminarse !!!



Ejemplo:

Supongamos que el conjunto de entidades Alumno se representa los alumnos que se encuentran doctorando. Entre el conjunto de entidades Alumno y Profesor, la relación ***DirigidoPor*** no es redundante. Indica cuál es el docente que dirige su trabajo de tesis y no puede inferirse de las materias que cursa.



Silvia Alicia Gómez & Leticia Irene Gómez

2.2 Relaciones Binarias vs. Ternarias

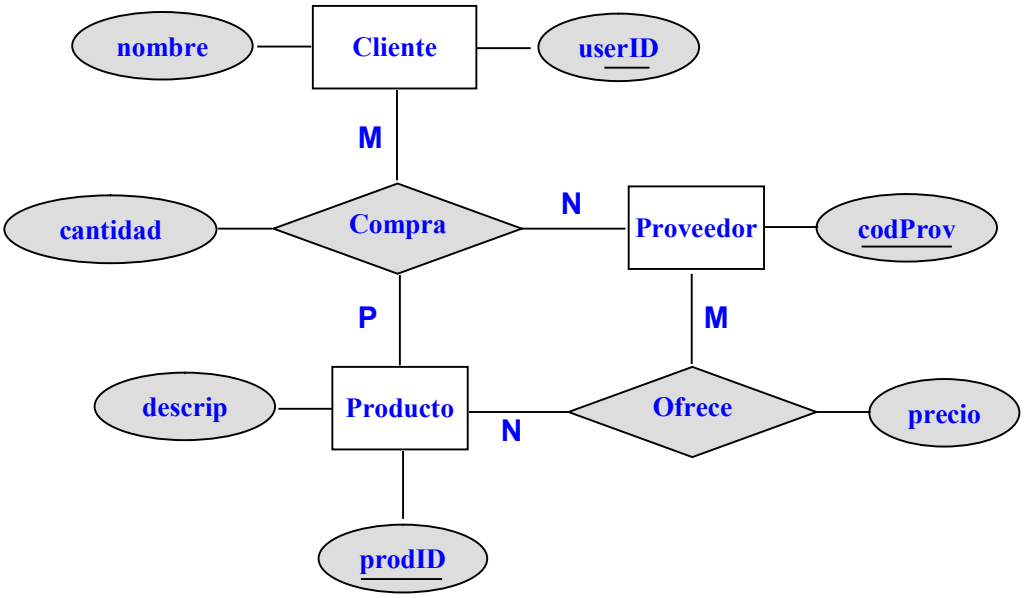
Puede siempre reemplazarse una relación ternaria por varias binarias y mantener la semántica? No siempre. El reemplazo puede ocasionar pérdida de información.

Ejemplo:

Sea un sitio como Amazon que vende productos inclusive de otros vendedores. Suponiendo un esquema no inflacionario (los precios no cambian en el tiempo).

- Escenario 1:

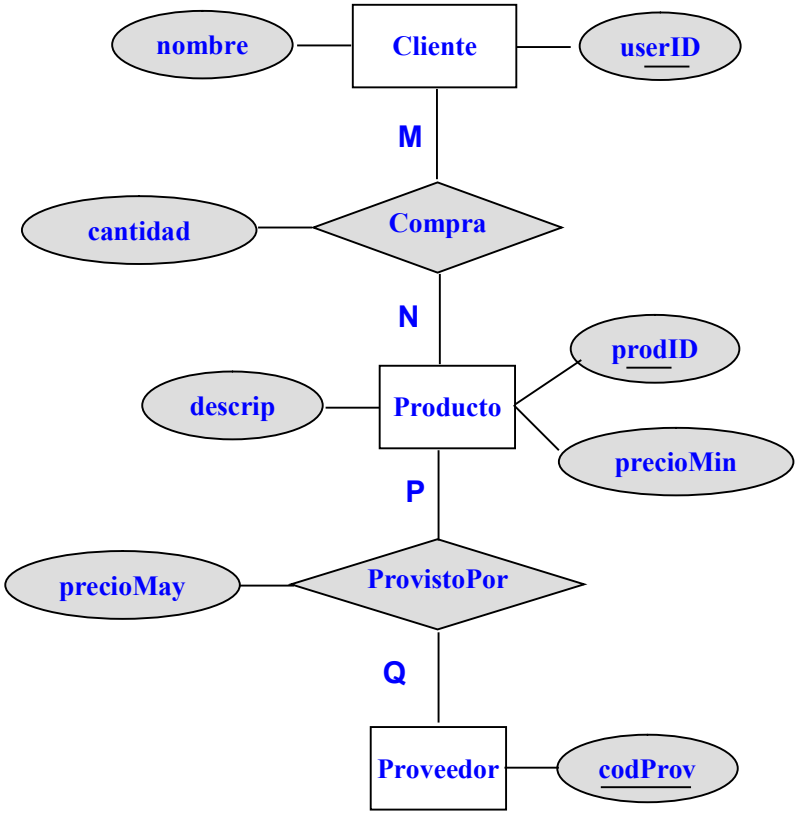
Amazon, cuando involucra a otros proveedores es sólo un intermediario y respeta el precio de venta que dichos proveedores ofrecen. Así, un cliente puede elegir comprar un DVD a diferentes proveedores vía Amazon, y a diferentes precios según cuál sea el proveedor. Si lo que quiere es poder representarse la información que el cliente paga por cada producto, como el precio depende del proveedor hay que utilizar una relación ternaria entre Cliente, Producto y Proveedor. La relación binaria Ofrece no es redundante.



- Escenario 2:

Si Amazon decidiera ocultar la información al cliente y vender el producto al mismo precio sin importar cuál sea la fuente, o sea ofrece el mismo producto siempre al mismo precio minorista, el escenario debe representarse con dos relaciones binarias: una entra entre Proveedor y Producto (seguramente registrando dicho precio del proveedor que no se visualiza en el cliente) y otra entre Cliente y Producto (con el precio minorista del producto). Lo que se quiere representar es el producto comprado y no a quien se lo compró internamente.

Silvia Alicia Gómez & Leticia Irene Gómez



Notar que aunque el atributo "precioMayorista" no se representara, estas 2 relaciones binarias no son equivalentes a una ternaria.

Silvia Alicia Gómez & Leticia Irene Gómez